Les Structures Répétitives en Algorithmique et Langage C : Boucles While, Do-While et For

Thème principal : Structures itératives en C : syntaxe, conditions d'arrêt, cas d'usage et évitement des boucles infinies

Synthèse exécutive : Ce cours détaille le fonctionnement, la syntaxe et les critères de choix des trois structures itératives fondamentales (while, do-while, for) en algorithmique et en langage C. L'objectif principal est de permettre l'automatisation du traitement répété d'instructions en évitant la duplication de code, conformément au principe DRY (Don't Repeat Yourself). Un accent particulier est mis sur la gestion rigoureuse des conditions d'entrée et de sortie, ainsi que sur la prévention des boucles infinies.

1. Utilité des structures itératives et principe DRY

En programmation, la répétition d'instructions ne doit jamais être gérée par copier-coller. Si l'on doit traiter 10 000 éléments d'un tableau ou redemander une saisie utilisateur, dupliquer le code détruit la maintenabilité, alourdit inutilement le programme et génère une source majeure de bogues. Les structures itératives (ou boucles) permettent de faire exécuter un même bloc d'instructions de façon répétée tant qu'une condition logique demeure vraie. L'un des risques majeurs associés est la 'boucle infinie' : si le bloc d'instructions ne modifie jamais les variables intervenant dans la condition d'arrêt, le programme se bloque indéfiniment.

- Principe DRY (Don't Repeat Yourself) : interdiction fondamentale de dupliquer du code pour exécuter plusieurs fois la même tâche
- Condition d'arrêt : expression booléenne évaluée pour déterminer la poursuite ou la sortie de la boucle
- Variable de contrôle / Compteur : variable devant être impérativement mise à jour au fil des itérations pour garantir la fin du processus

Cas concret enseignant : *Métaphore du tapis roulant d'usine d'emballage : tant qu'un colis est détecté par le capteur (condition vraie), le bras robotisé le prend et l'emballe (instruction). Si le capteur ne détecte plus de colis ou que le nombre maximal fixe de colis est atteint, le robot s'arrête. Sans condition de fin fonctionnelle, le robot continuerait à fonctionner à vide jusqu'à provoquer la surchauffe et la destruction de la machine.*

2. La boucle TANT QUE (while) : Structure à pré-condition

La boucle 'while' est la structure itérative par excellence lorsque le nombre précis d'itérations n'est pas déterminé à l'avance, mais dépend d'un événement dynamique (saisie au clavier, signal réseau, lecture de fichier). Puisque le test s'effectue dès l'entrée, si la condition est évaluée à faux initialement, les instructions à l'intérieur de la boucle sont totalement ignorées. Pour éviter une boucle infinie, il est impératif que le corps de la boucle modifie au moins une variable impliquée dans la condition booléenne.

- Pré-conditionnement : la condition d'accès est évaluée AVANT chaque passage dans le bloc d'instructions
- Intervalle d'itérations : de 0 à N itérations (le bloc peut ne jamais s'exécuter si la condition est fausse d'entrée)
- Initialisation préalable : toute variable testée dans la condition doit posséder une valeur définie avant l'instruction while

Cas concret enseignant : *Exemple de la vérification de code PIN à un distributeur de billets : la machine demande le code secret TANT QUE le code saisi est incorrect ET QUE le compteur d'essais est inférieur à 3. Si l'usager tape le bon code du premier coup, la boucle s'arrête immédiatement sans répéter la demande.*

3. La boucle FAIRE... TANT QUE (do-while) : Structure à post-condition

À la différence du 'while', la structure 'do-while' garantit que le corps de la boucle est exécuté au moins une fois, le contrôle ayant lieu en sortie du bloc. Elle est principalement appliquée aux contrôles de saisie (s'assurer qu'un nombre est compris dans un intervalle) et aux menus interactifs. En langage C, la syntaxe s'écrit `do { ... } while (condition);`. L'oubli du point-virgule final constitue une erreur de syntaxe courante soulevée au niveau de la compilation.

- Post-conditionnement : le test logique est réalisé APRÈS l'exécution du bloc d'instructions
- Garantie d'exécution : la boucle s'exécute au minimum 1 fois (intervalle de 1 à N itérations)
- Syntaxe C stricte : obligation d'apposer un point-virgule ';' après la parenthèse fermante de la condition while

Cas concret enseignant : *Cas du contrôle de saisie d'une note d'examen : 'Il est impossible de contrôler la validité de la saisie d'un étudiant avant qu'il n'ait tapé une valeur sur son clavier. On doit donc exécuter la saisie au moins une fois, ce qui fait du do-while la structure parfaitement adaptée.'*

4. La boucle POUR (for) : Itération déterminée

La structure 'for' regroupe en une ligne claire et centralisée l'ensemble de la mécanique d'itération. On l'emploie quand le nombre de répétitions est fixé par avance (ex: répéter 50 fois, parcourir un tableau). Bien que les trois expressions séparées par des points-virgules dans l'en-tête soient techniquement optionnelles en langage C, les omettre nuit gravement à la lisibilité et doit être évité.

- En-tête condensé : réunit initialisation, condition de maintien et incrémentation sous la forme `for(init; cond; inc)`
- Usage privilégié : réservée aux situations où le nombre d'itérations est connu avant d'entrer dans la boucle
- Portée (scope) : déclaration possible de la variable d'itération directement au sein de l'en-tête

Cas concret enseignant : *Métaphore de la course sur piste : vous savez avant de démarrer que vous devez effectuer exactement 5 tours. Vous préparez le compteur au tour 1 (initialisation), vous vérifiez que vous n'avez pas dépassé 5 (condition), et après chaque franchissement de ligne d'arrivée, vous ajoutez 1 au compteur (incrémentation).*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante ou bloque indéfiniment quand j'oublie d'incrémenter 'i' dans une boucle while ?
Réponse du professeur : Si la variable de contrôle 'i' n'est pas modifiée à l'intérieur du corps de la boucle, la condition de test (ex: i < 10) conserve constamment la même valeur vraie. La boucle s'exécute alors sans jamais pouvoir s'arrêter, accapare les ressources du processeur et provoque le gel apparent (plantage) du programme.

Question étudiant : Est-ce qu'on peut toujours remplacer une boucle 'for' par une boucle 'while' ?
Réponse du professeur : Oui. Toute boucle 'for' peut être réécrite équivalemment avec un 'while' en déplaçant l'initialisation avant la boucle et l'incrémentation en toute fin de bloc d'instructions. Cependant, la boucle 'for' reste la norme de lisibilité absolue dès lors que le nombre d'itérations est connu d'avance.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme complet en C qui calcule la somme des N premiers entiers positifs (ex: si N = 5, calculer 1 + 2 + 3 + 4 + 5 = 15). Le programme doit valider que N est un entier strictement positif (> 0) via une boucle 'do-while', puis calculer le cumul au moyen d'une boucle 'for'.

Résolution pas à pas :
Étape 1 : Déclaration des variables entières : 'n', 'somme' (initialisée à 0) et 'i'. Étape 2 : Saisie sécurisée du nombre N au moyen d'une boucle do-while : do { printf("Entrez N (>0) : "); scanf("%d", &n;); } while (n <= 0); Étape 3 : Calcul de la somme cumulative à l'aide d'une boucle for : somme = 0; for (i = 1; i <= n; i++) { somme += i; } Étape 4 : Affichage du résultat à l'aide de printf.

Résultat : Si l'utilisateur saisit N = 5, la boucle 'for' effectue 5 itérations : à i=1 (somme=1), i=2 (somme=3), i=3 (somme=6), i=4 (somme=10), i=5 (somme=15). Le programme affiche la valeur 15. Si l'utilisateur saisit

une valeur négative ou nulle, la boucle do-while redemande la saisie jusqu'à obtenir un entier valide.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'évaluation entre une boucle 'while' et une boucle 'do-while' ?

R : La boucle 'while' teste sa condition AVANT d'exécuter le bloc (de 0 à N itérations), alors que 'do-while' teste sa condition APRÈS l'exécution du bloc (de 1 à N itérations).

Q : Que se passe-t-il si la condition initiale d'une boucle 'for' est fausse dès le premier test ?

R : Le corps de la boucle n'est pas exécuté du tout (0 itération) et le programme enchaîne directement avec l'instruction qui suit la boucle.

Q : Quel oubli de syntaxe systématique provoque une erreur de compilation sur la boucle 'do-while' en C ?

R : L'oubli du point-virgule (;) obligatoire placé à la suite de la parenthèse de fermeture de la condition while : `do { ... } while(condition);`.